

Introduction to Scalability

Phase 1 · Fundamentals · Estimated time: 1.5 hours

1. Concept From First Principles

Scalability is a system's ability to handle growing load — more users, more requests, more data — without a proportional degradation in performance. There are exactly two ways to add capacity to a system, and almost every architecture decision in this program ultimately traces back to this choice.

It helps to think of these two options as sitting on a spectrum of operational complexity you take on in exchange for headroom. Every system starts simple, usually on a single machine, and the honest first question in an interview is never "how do I scale this to a billion users" — it's "at what point does this specific design actually need to change," a question your Chapter 14 estimation skills directly answer. Reaching immediately for maximum horizontal complexity on a system that will never see meaningful load is its own kind of design mistake, just a less commonly discussed one.

Vertical scaling (scale up)

Make the existing machine bigger: more CPU, more RAM, faster disks. It's simple — no code changes, no distributed-systems complexity — but it has a hard ceiling (there's a biggest machine you can buy), it's a single point of failure, and it usually costs disproportionately more as you approach that ceiling (a machine twice as powerful often costs far more than twice as much).

Horizontal scaling (scale out)

Add more machines and distribute load across them (using a load balancer, Chapter 9). This has effectively no ceiling — you can keep adding servers — and failure of one machine doesn't take down the whole system. The cost is complexity: now your application must work correctly when spread across many machines, which introduces the entire field of distributed systems (Phase 2 and 3 of this program).

Backend engineer analogy

Upgrading your production server from 4GB to 32GB RAM to handle more traffic is vertical scaling. Running your Laravel app on 10 separate servers behind a load balancer is horizontal scaling. Most experienced backend engineers have done vertical scaling instinctively (it's the easy first move); HLD interviews almost always expect you to reach for horizontal scaling as the real, durable answer.

The precondition for horizontal scaling: statelessness

Horizontal scaling only works cleanly if any server can handle any request — which means the application server itself should not store data that a later request depends on (like a session stored in local memory). A **stateless** service keeps all shared state externally — in a database, a cache like Redis, or a token (e.g., a JWT) — so it doesn't matter which server instance handles which request. This is why session data usually moves to Redis instead of PHP's default file-based session storage

the moment an app grows past one server.

Databases themselves are often the hardest part to scale horizontally, precisely because they inherently hold state — this is exactly why Phase 2 dedicates multiple chapters (Replication, Sharding, Consistent Hashing) specifically to scaling stateful systems.

In practice, horizontal scaling is rarely done manually by an engineer watching a dashboard — cloud platforms support **auto-scaling**: rules that automatically add or remove server instances based on live metrics like CPU utilization, request queue depth, or custom application metrics. A rule might read "if average CPU across the fleet exceeds 70% for 3 minutes, add 2 instances." This is what lets a well-designed stateless service handle Careem's Friday evening rush and quietly scale back down overnight without anyone manually intervening.

2. Real-World Example / Case Study

During Careem's peak commute hours or Talabat's Friday-night dinner rush, traffic can be several times normal levels. Their application tier scales horizontally — auto-scaling adds more stateless application server instances behind a load balancer as traffic rises, then removes them as traffic falls, keeping cost proportional to actual demand. This elasticity is only possible because the application servers are stateless; a new instance can start serving traffic immediately with zero warm-up of local state.

Meesho's catalog database, by contrast, cannot simply "add more servers" the same casual way — scaling a relational database horizontally requires deliberate techniques like read replicas and sharding, which is exactly why Phase 2 treats database scaling as a separate, deeper topic from application scaling.

A second example: Zomato/Swiggy's order-placement service scales horizontally and elastically across the lunch and dinner peaks, while their core restaurant-catalog database, which changes far less frequently and needs strong consistency for pricing and availability, has historically leaned more on vertical scaling and read replicas rather than aggressive horizontal partitioning — a reminder that different components of the same product often sit at very different points on this trade-off, based on their own read/write patterns and consistency needs.

3. Diagram

```
VERTICAL SCALING (scale up):
[ Server: 4 CPU, 8GB ] --> [ Server: 16 CPU, 64GB ]
Same single machine, just bigger. Ceiling exists. SPOF remains.

HORIZONTAL SCALING (scale out):
      [ Load Balancer ]
      /       |       \
[Server 1] [Server 2] [Server 3] ... [Server N]
Add more identical, stateless machines. No practical ceiling.
One server failing does not take down the system.
```

4. Trade-offs Table

Aspect	Vertical Scaling	Horizontal Scaling
Complexity	Low — no code changes needed	Higher — requires stateless design, coordination
Ceiling	Hard limit (biggest machine available)	Effectively unlimited
Failure impact	Single point of failure	One instance failing doesn't take down the system
Cost curve	Grows steeply near the top end	Roughly linear, pay for what you add
Typical first move	Quick fix for an unexpected spike	The durable, standard answer in HLD interviews

5. Common Interview Questions

Q1: "How would you scale this service to handle 10x traffic?"

Model approach: Default to horizontal scaling of the application tier behind a load balancer, explicitly confirm/ensure the service is stateless first (session data externalized), then separately address whether the database can keep up (caching first, then read replicas/sharding if needed) — don't conflate app-tier scaling with database scaling; they're different problems with different solutions.

Q2: "Why can't we just keep buying a bigger database server forever?"

Model approach: Explain the hard ceiling and steep cost curve of vertical scaling, and that it remains a single point of failure no matter how big it gets. Note that vertical scaling is still a legitimate first step (it's simple and often enough for a while) before investing in the added complexity of horizontal database scaling techniques.

Q3: "How would auto-scaling decide when to add or remove servers?"

Model approach: Describe metric-based rules (CPU, memory, request queue depth, or a custom business metric) with thresholds and a cooldown period to avoid rapidly flapping up and down. Mention that scaling out (adding servers) is usually made aggressive/fast to protect user experience during a spike, while scaling in (removing servers) is deliberately made more conservative/slow, to avoid prematurely shedding capacity during a brief lull.

6. Practice Exercises

- 1 Explain why a service storing user sessions in local server memory cannot be horizontally scaled without changes, and describe the fix.
- 2 For a system you've worked on, identify one component that was scaled vertically and one that would need horizontal scaling to keep growing.
- 3 Draw the horizontal scaling diagram from memory, including the load balancer.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"System Design Fundamentals: Vertical Scaling vs Horizontal Scaling Explained"	Focused, interview-oriented explanation.
English	"Horizontal and Vertical Scaling (How to Scale Your Application)"	Practical framing of when to use each.
Hindi	"Horizontal Vs Vertical Scaling System Design Simplified Hindi"	Uses real-world analogies (shops/buildings) in Hindi.
Hindi	"Horizontal or Vertical Scaling? Explained in Hindi" (Day 6/100 series)	Concise Hindi comparison as part of a 100-day prep series.

8. Key Takeaways

- Vertical scaling = bigger machine; horizontal scaling = more machines.
- Vertical scaling is simple but has a hard ceiling and remains a single point of failure.
- Horizontal scaling has no practical ceiling but requires the service to be stateless.
- Stateless services externalize shared state (DB, cache, tokens) so any instance can handle any request.
- Application-tier scaling and database scaling are different problems — don't conflate them in an interview answer.
- HLD interviews almost always expect horizontal scaling as the default, durable answer for growth.